



TED UNIVERSITY
Faculty of Engineering

Department of Computer Engineering

CMPE 492 Senior Project

ARIS

Low Level Design Report

Spring 2026

Taha Akdemir | ID: 12577008730

Ecem Nur Bilgi | ID: 14830754170

Şevkiye Sıla Kahya | ID: 14620089482

Elif Zeynep Elverişli | ID: 13942218426

Date	Reason For Changes	Version
19.03.2026	First Release	1.0.0
26.03.2026	Added detailed design artifacts including code snippets and UML diagrams	2.0.0

Table of Contents

1. Introduction	3
1.1 Object Design Trade-offs	3
1.1.1 Edge Processing vs. Cloud Processing.....	3
1.1.2 Lightweight Models vs. High-Accuracy Models	4
1.1.3 Multimodal Input vs. Single-Input System	5
1.1.4 Modularity vs. Runtime Performance	6
1.1.5 Data Privacy vs. System Monitoring.....	7
1.1.6 Request-Response Communication vs. Persistent Real-Time Communication.....	8
1.2 Interface Documentation Guidelines.....	9
1.2.1 API Contract Guidelines.....	9
1.2.2 Communication and Protocol Design Guidelines	12
1.2.3 Error Response and Handling Guidelines	15
1.2.4 Internal Component and Class Interface Guidelines.....	18
1.2.5 Interface Documentation and Specification Format.....	20
1.3 Engineering Standards.....	21
1.3.1 Documentation and Modeling Standards	22
1.3.2 Code Quality and Object-Oriented Design Standards.....	23
1.3.3 Security, Reliability, and Operational Standards.....	24
1.3.4 Development and Verification Practices	25
1.3.5 Summary of Standards	25
1.4 Definitions, Acronyms, and Abbreviations	26
1.4.1 Core Definitions	26
1.4.2 Acronyms and Abbreviations	27
1.4.3 Assumptions	28
2. Packages	29
2.1 Input Acquisition and Buffering Package	29

2.2 Recognition and Interpretation Package	31
2.3 Command Processing and Communication Package	32
2.4 Security, Monitoring, and Administration Package.....	34
3. Class Interfaces	37
3.1 Input Processing Component	37
3.1.1 CameraManager	37
3.1.2 MicrophoneManager	38
3.1.3 InputBuffer	38
3.2 Recognition Component.....	39
3.2.1 GestureRecognizer	40
3.2.2 FaceExpressionRecognizer	40
3.2.3 VoiceCommandRecognizer	41
3.3 Command Processing Component	42
3.3.1 CommandValidator.....	42
3.3.2 CommandMapper	43
3.3.3 CommandDispatcher	43
3.4 Communication Component	45
3.4.1 CommunicationManager	45
3.4.2 ConnectionHandler.....	46
3.4.3 MessageBuffer.....	47
3.5 Security Component.....	48
3.5.1 AuthenticationManager	49
3.5.2 EncryptionManager	50
3.5.3 AccessControlManager	51
3.6 Web Interface and Administration Component	52
3.6.1 WebInterfaceController	52
3.6.2 RoleManager	53
3.6.3 LogManager	54
4. Glossary.....	57
5. References	61

1. Introduction

1.1 Object Design Trade-offs

The object design phase required multiple trade-offs as the team refined the initial architectural framework into a usable low-level design. These trade-offs were evaluated by considering system requirements, operational demands, reliability, privacy constraints, and hardware limitations. The final design decisions affected both system architecture and the distribution of tasks between system components and interface elements and solution components.

1.1.1 Edge Processing vs. Cloud Processing

Chosen Approach: Edge processing with on-device AI inference

The ARIS system requires operation in situations where uninterrupted network access cannot be assured. The core recognition tasks which include gesture and voice and facial expression recognition operate directly on the helmet unit because of this requirement which prevents remote processing through cloud services.

Advantages:

- Communication delays decrease which results in improved system response times
- The system maintains functionality when network connections become unstable or completely fail
- The system protects user data by not sending unprocessed biometric information
- The system achieves greater operational autonomy in field conditions.

Disadvantages:

- Edge devices have restricted processing capacity
- The system requires both energy efficient processing pathways and efficient lightweight system solutions

Alternative Considered: Cloud-based processing

- **Pros:** Greater computational capacity, easier deployment of larger models
- **Cons:** Higher latency, dependence on connectivity, increased privacy risks

Reason for Rejection:
Cloud-based processing was not selected because the operational context of ARIS requires low-latency and reliable behavior even in communication-constrained scenarios. This decision also shaped the low-level design by placing components such as local input handlers, preprocessing units, and recognition modules on the helmet side, while assigning monitoring and storage-related responsibilities to backend-side components.

1.1.2 Lightweight Models vs. High-Accuracy Models

Chosen Approach: Lightweight and efficient models suitable for edge deployment

Since ARIS performs inference on device-side hardware, model efficiency was prioritized during object design. The system requires models which can deliver precise results inside the operating constraints of the helmet unit.

Advantages:

- Faster inference time
- Lower CPU, memory, and energy consumption
- Better suitability for embedded or edge-oriented hardware

Disadvantages:

- The model achieves lower recognition accuracy which is smaller than the accuracy of more complex models
- The system shows diminished ability to process extremely intricate input patterns

Alternative Considered: High-accuracy, resource-intensive models

- **Pros:** The system can achieve improved recognition performance for users

- **Cons:** The system requires additional time to complete tasks while using more computational resources which decreases its ability to function in real-time

Reason

for

Rejection:

The design goal of ARIS is to provide timely assistance in operational settings. Therefore, marginal gains in recognition accuracy were considered less critical than predictable and fast response times. The decision at the object design level determined how recognition components would function through their interfaces and internal tasks which operated to achieve optimized inference pipelines instead of supporting large and computationally intensive models.

1.1.3 Multimodal Input vs. Single-Input System

Chosen Approach: Multimodal input handling (voice, gesture, and facial expression)

ARIS was built as a multimodal communication system to boost its operational reliability and user-friendly design across different usage scenarios. Environmental noise and restricted movement and lighting conditions and partial sensor failure all create situations where a single modality becomes untrustworthy. The system achieves better communication support through its ability to process multiple input channels.

Advantages:

- Increases robustness through redundancy
- Improves usability under varying environmental conditions
- Reduces dependence on a single sensing mechanism
- Supports more flexible interaction patterns

Disadvantages:

- The system requires more time to implement because of its complex system setup.
- The system needs better coordination between its input mechanisms and processing methods.

Alternative Considered: Single-modality systems

- **Pros:** Simpler implementation, fewer dependencies
- **Cons:** Lower fault tolerance and reduced reliability in changing conditions

Reason

for

Rejection:

A single input modality was considered insufficient for the intended use case of ARIS. This decision directly affected the low-level design by requiring separate but coordinated solution components for visual, audio, and expression-based input processing, along with an integration mechanism for command interpretation.

1.1.4 Modularity vs. Runtime Performance

Chosen Approach: Balanced modular design

The system uses modular decomposition which helps to create better maintenance solutions and better extension capabilities and better understanding of responsibility boundaries. The system development process enables the team to incrementally develop system components which represent its main functions through separate testing and refinement activities.

Advantages:

- Easier maintenance and debugging
- Better support for future extension and reuse
- Clear separation of responsibilities
- More manageable subsystem and interface design

Disadvantages:

- Additional communication overhead between modules
- Slight runtime cost due to abstraction boundaries

Alternative Considered: Monolithic design

- **Pros:** Lower communication overhead, potentially faster execution
- **Cons:** Difficult to maintain, test, and extend

Reason	for	Rejection:
ARIS needs a modular design because it will develop through several sensing and communication technologies. A modular structure better supports both current implementation and future refinement. The design process creates a trade-off which shows component interfaces and component responsibilities through separate elements that handle input processing and communication and logging and user interface control.		

1.1.5 Data Privacy vs. System Monitoring

Chosen Approach: Privacy-preserving logging and monitoring

The ARIS system needs to keep all user input data completely protected because it handles special multimodal user information which includes biometric data and behavioral signals. The system logs events which have been generated through execution together with their corresponding command results and operational metadata which has been anonymized.

Advantages:

- Protects sensitive user data
- Reduces ethical and security risks
- Aligns with privacy-conscious system design principles

Disadvantages:

- Limits the amount of data available for debugging and retrospective analysis

Alternative Considered: Full raw-data logging

- **Pros:** More detailed diagnostics and model analysis
- **Cons:** Significant privacy, security, and ethical concerns

Reason	for	Rejection:
Storing raw biometric or sensory data would introduce unnecessary privacy risks and increase the security burden of the system. Therefore, the object design reflects a controlled logging strategy in		

which monitoring-related components operate on metadata and event summaries rather than raw image, audio, or expression data.

1.1.6 Request-Response Communication vs. Persistent Real-Time Communication

Chosen Approach: Primarily request-response based communication

For the current system scope, ARIS uses a command-oriented communication model in which recognized commands, acknowledgements, and status data are exchanged as discrete interactions. This approach simplifies integration between the helmet-side components and backend services.

Advantages:

- Simpler implementation and interface design
- Easier integration with backend services
- Better maintainability for the current prototype scope
- Supports stateless service interactions where appropriate

Disadvantages:

- Less suitable for continuous push-based real-time updates
- May introduce minor delays if frequent polling is required

Alternative Considered: Persistent bidirectional communication (e.g., WebSocket-based messaging)

- **Pros:** Better support for real-time streaming and continuous updates
- **Cons:** Greater implementation complexity, more difficult connection management

Reason	for	Rejection:
Persistent real-time communication was not selected for the current version because ARIS primarily operates through event-driven command transfer rather than continuous media streaming. A simpler request-response model was considered sufficient for the prototype while keeping the		

design easier to implement and test. However, persistent bidirectional communication may be considered in future versions if the system scope expands toward continuous live updates.

1.2 Interface Documentation Guidelines

To ensure consistency, maintainability, interoperability, and ease of future extension, all interfaces in the ARIS system are defined and documented according to standardized guidelines. These guidelines apply not only to external communication interfaces, such as backend APIs and dashboard endpoints, but also to internal subsystem and class-level interfaces. The purpose of this documentation approach is to provide a clear and consistent specification of how system components exchange data, invoke operations, handle errors, and enforce security constraints.

1.2.1 API Contract Guidelines

The API contracts define how ARIS helmet units, backend services, and the web-based monitoring dashboard communicate with each other. These contracts are designed to support reliable integration, secure data exchange, and long-term maintainability.

1. Base URL Structure

- A consistent base URL shall be used for all backend API endpoints.
- Example: `https://aris-server/api/v1`
- API versioning shall be included in the URL structure to support future updates without breaking compatibility with existing clients.

2. Endpoint Naming Conventions

- Endpoint names shall use lowercase letters and resource-oriented naming conventions.
- Examples: `/commands`, `/helmets/{id}/status`, `/logs`, `/sessions/{id}`
- Naming shall remain descriptive, consistent, and aligned with the responsibilities of the related subsystem.

3. HTTP Methods

Appropriate HTTP methods shall be selected based on operation semantics:

- **GET:** Retrieve system information such as helmet status, logs, or dashboard data
- **POST:** Submit commands, telemetry data, authentication requests, or acknowledgements
- **PUT:** Update configuration settings or command mappings
- **DELETE:** Remove temporary or inactive records when such functionality is required

4. Request and Response Formats

- JSON shall be used as the standard format for request and response bodies.
- Field names shall be clear, descriptive, and consistently named across endpoints.
- A command-related request may include fields such as:
 - timestamp
 - device_id
 - session_id
 - command_type
 - confidence_score
 - payload
 - priority

5. Response Consistency

- All successful responses shall follow a consistent structure wherever applicable.
- Typical response fields may include:
 - status
 - message
 - data
 - timestamp
 - request_id

Example Command Request Body:

```
{
  "timestamp": "2026-03-25T10:30:00Z",
  "deviceId": "helmet_01",
  "sessionId": "sess_204",
  "commandType": "MOVE_FORWARD",
  "confidenceScore": 0.87,
  "payload": {
    "direction": "forward",
    "speed": "medium"
  },
  "priority": "HIGH"
}
```

This example illustrates a typical command request sent from the helmet device to the backend system.

Example Command Response Body:

```
{
  "status": "success",
  "message": "Command received and processed successfully.",
  "data": {
    "commandId": "cmd_7812",
    "executionState": "accepted"
  },
  "timestamp": "2026-03-25T10:30:01Z",
  "requestId": "req_4519"
}
```

This example shows the structured response returned by the backend after processing a command.

6. Authentication and Authorization

- Secure authentication shall be required before communication between helmet units and backend services is established.
- Authorized users shall access the monitoring dashboard through secure login mechanisms.
- Access to logs, configuration settings, and administrative operations shall be restricted according to user roles and permissions.

1.2.2 Communication and Protocol Design Guidelines

The ARIS system needs to function within environments that experience both dynamic changes and potential stability issues. The system uses communication interfaces which provide to achieve dependable system operation between helmet devices and backend systems and monitoring centers.

1. Request-Response Oriented Communication

- The communication system uses request-response methods as its main interaction method between different parties.
- The request must include all essential details needed for proper execution according to the request requirements.
- The system needs to use explicit session identifiers because it demands continuous operation instead of depending on automatic system state detection.
- This approach improves scalability, simplifies failure recovery, and reduces coupling between communicating components.

2. Session Awareness

- The system reduces unnecessary state dependence through its design but needs to enable controlled session management when authentication and tracking and command continuity require it.
- Session-related information shall be explicitly represented using identifiers such as `session_id`.

- The centralized session handling system requires to maintain its operational standard through complete documentation.

3. Request Control and Rate Limiting

- The system needs to implement request control methods which prevent devices and dashboard users from generating excessive system traffic.
- The system allows users to set a specific request cap which applies to each device and session and user account.
- The system temporarily stops or delays incoming requests when they exceed established thresholds to protect system stability.

4. Data Integrity and Validation

- All incoming data shall be validated before processing.
- Validation shall include:
 - required fields must be present
 - data types must match correct values
 - acceptable value ranges
 - related fields must show logical relationships
- Invalid data shall be rejected with clear and structured error responses.

5. Connection Resilience and Recovery

- The communication design shall tolerate temporary network interruptions.
- If transmission fails, command-level data generated on the helmet may be buffered locally.
- Buffered data may be retransmitted when connectivity is restored.

Example Request Processing Flow:

```
public class RequestProcessor {  
  
    public String processRequest(CommandRequest request) {  
        if (request == null) {  
            return "Invalid request";  
        }  
  
        if (!isValid(request)) {  
            return "Request validation failed";  
        }  
  
        if (!isSessionActive(request.getSessionId())) {  
            return "Session not found";  
        }  
  
        if (isRateLimitExceeded(request.getDeviceId())) {  
            return "Too many requests";  
        }  
  
        return "Command processed successfully";  
    }  
  
    private boolean isValid(CommandRequest request) {  
        return request.getDeviceId() != null  
            && request.getSessionId() != null  
            && request.getCommandType() != null;  
    }  
  
    private boolean isSessionActive(String sessionId) {  
        return sessionId != null && !sessionId.isEmpty();  
    }  
  
    private boolean isRateLimitExceeded(String deviceId) {  
        return false;  
    }  
}
```

This example demonstrates how incoming requests are validated, checked for session status, and processed within the system.

6. Secure Data Transmission

- All communication shall be performed over secure and encrypted channels.

- The system will only send processed command-level data and monitoring data during standard operations.
- Sensitive raw inputs such as audio streams, raw image frames, and unprocessed biometric data shall not be exposed through standard communication interfaces.

1.2.3 Error Response and Handling Guidelines

The ARIS system defines structured and consistent error responses in order to support reliable communication, easier debugging, and safer recovery from operational failures. Error handling is treated as a core part of interface design rather than a secondary implementation detail.

1. Standard Error Response Structure

All error responses shall follow a consistent JSON structure. An error response shall include:

- status
- message
- code
- timestamp
- request_id

2. Standard Error Categories

Standard status codes and categories shall be used for common failure scenarios:

- **400 – Bad Request:** Missing or invalid request fields
- **401 – Unauthorized:** Authentication failure
- **403 – Forbidden:** Access to restricted resources
- **404 – Not Found:** Requested resource, session, or entity does not exist
- **408 – Request Timeout:** Communication delay or timeout
- **429 – Too Many Requests:** Rate limit exceeded
- **500 – Internal Server Error:** Unexpected backend or processing failure

3. Command Processing Errors

Errors related to command interpretation and validation shall be handled explicitly. Such cases may include:

- low confidence score in recognition
- unsupported or unknown command
- incomplete multimodal input
- inconsistent command mapping results

In such cases, the system may reject the command, request re-evaluation, or return feedback indicating that the command could not be safely executed.

4. Communication and Connectivity Errors

The system shall handle network-related failures gracefully, including:

- temporary connection loss
- delayed transmission
- interrupted sessions
- partial delivery failures

Depending on the situation:

- commands may be buffered
- retry mechanisms may be triggered
- the user or dashboard may be notified
- the operation may be marked as pending or failed

Example Error Response Body:

```
{
  "status": "error",
  "message": "Low confidence score. Command cannot be executed safely.",
  "code": 400,
  "timestamp": "2026-03-25T10:32:14Z",
  "requestId": "req_5621"
}
```

This example presents a standard error response generated when a command cannot be processed successfully.

Example Command Error Handling:

```
public class CommandErrorHandler {

    public String handleCommand(CommandRequest request) {
        if (request == null) {
            return "Missing request data";
        }

        if (request.getConfidenceScore() < 0.50) {
            return "Low confidence score";
        }

        if (request.getCommandType() == null) {
            return "Unsupported or unknown command";
        }

        return "Command is valid";
    }
}
```

This example illustrates how command-related errors are detected and handled within the system logic.

5. User-Friendly and Actionable Messages

- Error messages shall be clear, concise, and actionable.
- Messages shall guide users or system components toward appropriate corrective action.

- Ambiguous or overly technical messages shall be avoided unless they are specifically intended for developer-facing logs.

1.2.4 Internal Component and Class Interface Guidelines

In addition to external APIs, the ARIS system contains internal interfaces between subsystems and solution-level components. These interfaces must also be documented consistently in order to support implementation, testing, and maintenance.

1. Method-Level Documentation

Each class-level or component-level interface shall document:

- method or operation name
- purpose and responsibility
- input parameters and data types
- return values and output format
- possible exceptions or failure conditions

2. Preconditions and Postconditions

- Important operations shall specify their required preconditions and expected postconditions.
- For example, a command validation operation may require a recognized command object as input and may produce either a validated command or an error state as output.

3. Data Type and Object Consistency

- Shared objects exchanged between components shall follow consistent naming and structure.
- Common message or data-transfer objects shall be reused where appropriate to reduce ambiguity and duplication.

4. Responsibility Boundaries

- Each interface shall reflect the single, well-defined responsibility of the related class or subsystem.
- Interfaces shall avoid exposing unnecessary internal details of a component.
- This supports modularity, maintainability, and controlled coupling.

5. Error Propagation Rules

- Internal interfaces shall define how errors are propagated, handled, or transformed between components.
- Recoverable and non-recoverable failures shall be distinguished where necessary.
- Errors shall not be silently ignored unless explicitly justified by the design.

Example	Internal	Component	Interaction:
---------	----------	-----------	--------------

```
public class MultimodalProcessor {  
    public CommandResult analyzeInput(AudioInput audio, ImageFrame image) {  
        String textCommand = convertSpeechToText(audio);  
        String gestureCommand = detectGesture(image);  
  
        if (textCommand == null && gestureCommand == null) {  
            return new CommandResult("No valid command detected", false);  
        }  
  
        String finalCommand = mergeCommands(textCommand, gestureCommand);  
  
        return new CommandResult(finalCommand, true);  
    }  
  
    private String convertSpeechToText(AudioInput audio) {  
        return "MOVE_FORWARD";  
    }  
  
    private String detectGesture(ImageFrame image) {  
        return null;  
    }  
  
    private String mergeCommands(String textCommand, String gestureCommand) {  
        if (textCommand != null) {  
            return textCommand;  
        }  
        return gestureCommand;  
    }  
}
```

This example represents how multimodal inputs are processed and combined to produce a final command.

1.2.5 Interface Documentation and Specification Format

To maintain clarity and ease of development, all interfaces in ARIS shall be documented using a consistent structure. This documentation format supports both implementation and future integration efforts.

1. Standard Documentation Structure

Each external or internal interface specification shall include the following elements where applicable:

- **Description:** Purpose of the interface
- **Owner Component/Class:** The subsystem or class responsible for the interface
- **Operation/Method Name or Endpoint:** The specific interaction being documented
- **Input Parameters / Request Body:** Fields, types, and meanings
- **Output / Response Body:** Returned values and their meanings
- **Preconditions:** Conditions that must hold before invocation
- **Postconditions:** Expected outcome after successful completion
- **Error Conditions:** Possible failure scenarios and corresponding codes or exceptions

2. Use of Documentation Tools

- External API documentation may be generated or maintained using tools such as Swagger or Postman.
- Internal interfaces may be represented using UML class diagrams, method tables, or structured interface specification tables.
- The selected representation shall prioritize readability and traceability to the system design.

3. Clarity and Readability

- Documentation shall use clear and consistent terminology aligned with the overall ARIS design.
- Technical explanations shall remain concise but sufficiently descriptive.
- Representative examples shall be provided for major interfaces whenever useful.

4. System-Specific Interface Descriptions

Documentation shall reflect the specific communication and processing flow of ARIS, including:

- command transmission from helmet units
- backend validation and processing
- dashboard retrieval of monitoring and log data
- internal interaction between recognition, validation, mapping, communication, and logging components

5. Maintainability and Version Tracking

- Interface documentation shall be updated whenever relevant changes are introduced.
- Version changes shall be clearly reflected in both external API specifications and internal design documents.
- Deprecated interfaces shall be marked explicitly to avoid misuse during future development.

Representative code snippets and examples provided in this section are intended for illustrative purposes and may evolve during the implementation phase.

1.3 Engineering Standards

The section defines the engineering standards together with best practices which were used to execute low-level design work and build and maintain the ARIS system. The selected standards will guarantee that the project produces complete documentation while maintaining modular design and system security and easy maintenance work, which meets the requirements of a real-time multimodal communication prototype.

ARIS operates as a distributed system which combines edge devices with backend services and communication modules and a web-based monitoring interface instead of functioning as a basic standalone application. The project needs to use engineering standards which will help them achieve software quality together with essential system performance requirements. The project uses documentation standards together with object-oriented design principles and secure communication practices and structured modeling techniques to achieve its objectives.

1.3.1 Documentation and Modeling Standards

The documentation of ARIS follows a structured process which enables traceable documentation according to IEEE system design standards. The objective of the project requires all deliverables to maintain clear and complete content which remains consistent throughout their development from requirements through analysis and high-level and low-level design work.

The project documentation reflects the following design artifacts:

- Class and object models to describe the relationships among core software components such as InputManager, RecognitionEngine, CommandMapper, CommunicationModule, and LogManager.
- Dynamic models and activity diagrams to represent the operational flow of multimodal input processing validation secure transmission and feedback handling
- Deployment and hardware/software mapping diagrams to show the distribution of software responsibilities between the Raspberry Pi-based helmet unit and the central server
- User interface mock-ups and navigational models to document the structure and intended use of the monitoring dashboard

These modeling choices are already consistent with the previous project documents, where the system has been represented through object/class structures, processing pipelines, activity diagrams, and dashboard navigation designs.

In terms of notation, the project adopts **UML-based modeling** as the primary visual documentation standard. In particular:

- **Class Diagrams** are used to represent the detailed low-level structure of system classes and interfaces
- **Activity Diagrams** are used to capture the internal workflow of command recognition and transmission

This approach ensures that the low-level design remains readable not only for the current team but also for future developers who may extend or maintain the system.

1.3.2 Code Quality and Object-Oriented Design Standards

ARIS uses fundamental object-oriented design principles together with modular design methods to create code that remains maintainable and extends beyond its original scope. The system design uses distinct responsibility areas which create separate operational functions. For example:

- The InputManager component handles the task of obtaining unprocessed sensor data from input sources
- The VisionProcessor and AudioProcessor components handle their respective media streams through their dedicated preprocessing and recognition functions
- The RecognitionEngine processes recognition results to generate potential command outputs.
- The CommandValidator component assesses both confidence levels and command structure components.
- The CommandMapper component transforms confirmed input data into a standardized command format.
- The CommunicationModule component enables safe data transmission between systems.
- The LogManager, SessionManager, UserManager, and ConfigManager components enable system monitoring and user authentication and administrative functions and system maintenance activities.

The modular class structure represents the organizational divisions already established in the Analysis Report and High-Level Design.

The low-level design uses these principles for its development:

- **Single Responsibility Principle (SRP):** Single Responsibility Principle requires that every class should perform one main function.
- **Open/Closed Principle (OCP):** The system should support extension, such as adding new commands or updating recognition logic, without requiring major restructuring.
- **Interface Segregation Principle (ISP):** Components should depend only on the interfaces relevant to their own responsibilities.
- **Dependency awareness:** Higher-level modules should interact with abstractions and clearly defined interfaces rather than tightly coupled implementations.

1.3.3 Security, Reliability, and Operational Standards

Since ARIS processes multimodal biometric-related inputs and transmits operational commands, engineering quality in this project also includes security and reliability standards, not only software structure.

The system design incorporates the following operational engineering standards:

- **Mutual authentication** between helmet units and the backend before communication begins
- **Role-Based Access Control (RBAC)** for separating permissions of helmet users, command center operators, and root users
- **HTTPS-only server interfaces** and **TLS 1.3-protected channels** for secure communication
- **AES-256 encryption** for command and feedback transmission
- **Secure logging and auditing** for traceability without violating privacy constraints

These are not abstract goals; they are already reflected in the High-Level Design and Specification documents.

From a professional engineering standpoint, these standards support a prototype that is secure, explainable, and maintainable within the limits of an academic implementation.

1.3.4 Development and Verification Practices

To improve implementation quality, the project also follows practical software engineering habits during development.

These include:

- use of version control for tracking code and documentation changes
- peer review within the team before merging major updates
- component-level testing of recognition, communication, and dashboard-related modules
- verification of interface consistency between edge, backend, and dashboard components

Because ARIS contains multiple interacting subsystems, development quality depends not only on whether each module works individually, but also on whether the interfaces between them remain stable and understandable. Therefore, maintainable documentation and interface verification are treated as engineering standards alongside implementation.

1.3.5 Summary of Standards

The engineering standards establish requirements which ARIS must meet to obtain the following results:

- technical documentation that shows all details through its structured model and UML design system
- an implementation design which developers can easily maintain and expand through its use of object-oriented modular design
- an operation system which provides safe and trusted performance through its use of encryption and authentication and access control and selective data retention
- development quality that meets industry standards through testing requirements and documentation practices and interface design consistency

The project uses these standards to create a complete low-level design which will function as a basis for developing a real-time multimodal communication prototype.

1.4 Definitions, Acronyms, and Abbreviations

1.4.1 Core Definitions

1. ARIS **System:**

The ARIS System functions as a real-time multimodal communication system which assists users by converting their voice, gesture, and facial expressions into structured commands which the system delivers to a central system through secure transmission.

2. Helmet Unit (Edge Device):

The Helmet Unit (Edge Device) functions as a wearable device which operates on Raspberry Pi technology to collect multimodal data and process it locally while sending authenticated commands to the backend system.

3. Backend **Server:**

The central system component responsible for command validation, session management, logging, and coordination between system modules and the monitoring dashboard.

4. Multimodal **Input:**

The combination of multiple input types such as voice commands, hand gestures, and facial expressions used to improve system robustness and reliability.

5. Recognition **Engine:**

The Recognition Engine is the component responsible for interpreting multimodal input data and generating a command candidate with an associated confidence score.

6. Command **Mapping:**

Command Mapping converts recognized inputs into predefined structured commands which the system can transmit and execute.

7. Command

Validation:

The process of verifying the correctness, confidence level, and completeness of a recognized command before transmission.

8. Monitoring

Dashboard:

The Monitoring Dashboard serves as a web-based interface which displays system status and command logs and device activity and operational metrics during real-time operation.

9. Session

Management:

Session Management functions as the system which monitors active device connections together with their communication states and authentication sessions that exist between the helmet unit and backend server.

10. Telemetry

Data:

Telemetry Data consists of operational information which includes connection status and system health data and device performance metrics that support monitoring and diagnostic activities.

1.4.2 Acronyms and Abbreviations

1. API (Application Programming Interface):

This protocol framework enables helmet devices to communicate with backend services and monitoring dashboard systems.

2. REST (Representational State Transfer):

A communication architecture commonly used for designing scalable request-response based API interactions.

3. UML (Unified Modeling Language):

The modeling language provides designers with tools to create system designs which include class diagrams and activity diagrams and deployment diagrams.

4. RBAC (Role-Based Access Control):

This security system controls access to computer systems according to different user roles which include helmet user and operator and administrator roles.

5. TLS (Transport Layer Security):

This cryptographic protocol protects data transmission security between different system elements.

6. HTTPS (HyperText Transfer Protocol Secure):

This protocol provides a secure method for HTTP to transmit encrypted data between clients and backend servers.

7. AES (Advanced Encryption Standard):

This symmetric encryption standard secures data that is sent over networks.

8. IoT (Internet of Things):

This system design enables physical devices to connect with central services through network communication which includes the helmet unit.

9. SRP (Single Responsibility Principle):

This design principle states that every component needs to handle a single specific task.

1.4.3 Assumptions

1. Hardware

Platform:

The helmet unit operates on a Raspberry Pi.

2. Connectivity

Conditions:

The system may operate under unstable or intermittent network conditions, requiring buffering and retry mechanisms.

3. Model

Constraints:

The use of lightweight recognition models enables real-time operations on edge devices.

4. User

Interaction:

Users interact with the system through predefined commands rather than unrestricted natural language.

5. Security

Requirements:

The system needs secure communication channels and authenticated devices to operate its complete functionality.

2. Packages

To retain modularity, maintainability, and a clear separation of responsibilities among system components, the ARIS low-level design is divided into packages. The layered and component-based structure previously developed in the earlier project reports is reflected in the package breakdown. The package structure must provide server-side monitoring and management services as well as edge-side processing on the helmet unit because ARIS is a distributed multimodal communication system rather than a single-process application.

Packages are used to organize related classes that work closely together and have a technical goal throughout the low-level design stage. This arrangement makes the design easier to read, streamlines implementation planning, and lessens entanglement between unrelated system components. Additionally, it enables future developers to expand the system in a controlled way without simultaneously affecting every component.

ARIS's package structure is built on the system's primary operating flow. Multimodal input is often captured by the system, which then uses recognition services to understand it, converts it into validated instructions, securely communicates these commands, and displays system information via administrative and monitoring interfaces. This order is already in accordance with the subsystem breakdown shown in the High-Level Design Report and the processing pipeline shown in the Analysis Report.

2.1 Input Acquisition and Buffering Package

The raw multimodal input data produced on the helmet unit is collected, temporarily stored, and synchronized by this program. It serves as the basis for all subsequent recognition and command processing operations and is the initial operational layer of the ARIS pipeline.

The kit contains parts like the helmet camera and microphone that communicate directly with input devices. Its primary responsibility is to guarantee that the system consistently receives legitimate and processable input streams under real-time operational conditions, rather than to comprehend user intent. Since ARIS is designed to operate in dynamic and perhaps unstable field conditions, this package needs to perform consistently even in the face of noise, mild illumination change, and brief disruptions in one of the input channels.

The main responsibilities of this package are as follows:

- initializing and managing the camera and microphone devices
- capturing visual and audio data continuously
- monitoring the availability and status of input devices
- temporarily buffering captured data
- synchronizing multimodal input into a processable package for the recognition layer
- supporting partial-input conditions when one modality is unavailable

The package contains the following classes:

- **CameraManager**
- **MicrophoneManager**
- **InputBuffer**

Initializing the camera, verifying its availability, and continually gathering visual frames from the surroundings are all under the purview of the CameraManager. Similar functions are carried out for audio input by the MicrophoneManager, which initializes the microphone, initiates and terminates recording, and makes the most recent audio segment accessible. By gathering frame and audio data, temporarily storing it, and creating synchronized input packages for subsequent identification, the InputBuffer serves as the package's coordination point.

Because low-latency data capturing is crucial to the system as a whole, this package is implemented on the helmet-side edge device. Sending unprocessed audio and video streams to a distant backend for first processing would raise latency, raise privacy issues, and go against the edge-processing architectural choice that was previously made. As a result, the classes included in this package allow real-time command recognition directly and function locally.

Dependency-wise, this package does not rely on higher-level administrative or decision-making services, although it does supply data to the recognition-related package. This one-way dependence

adheres to the responsibility separation outlined in the previous reports while maintaining modularity.

2.2 Recognition and Interpretation Package

The provided multimodal input must be analyzed by this program in order to extract identified user intent. It stands for the edge-side software's intelligence-focused component and acts as a link between unstructured sensory data and organized command-level comprehension.

The Input Acquisition and Buffering Package provides synchronized input data to the Recognition and Interpretation Package. The input is then compared to a predetermined gesture, facial expression, or vocal command using machine learning or rule-based recognition techniques. Every recognition process yields a comprehensible output that may subsequently be verified and converted into a system-level action.

Its primary duties consist of:

- loading and maintaining AI-based recognition models
- processing visual input for gesture recognition
- processing facial input for expression analysis
- processing audio input for predefined voice command detection
- producing recognized results that represent possible user intent
- supporting multimodal robustness by allowing recognition from different sensing channels

The package contains the following classes:

- **GestureRecognizer**
- **FaceExpressionRecognizer**
- **VoiceCommandRecognizer**

To recognize specific hand or body motions, the GestureRecognizer analyzes visual frames. The FaceExpressionRecognizer translates silent orders based on expressions by analyzing visual data pertaining to the user's face. After processing audio input, the VoiceCommandRecognizer compares

spoken information to a limited and predetermined collection of command patterns. In line with the multimodal structure specified in the Analysis Report and Low-Level Design trade-offs, these three recognizers reflect distinct recognition routes.

Because input management and recognition have distinct roles, this package is distinct from input capture. While the recognition package deals with deriving meaning from the data, the input package is focused on acquiring useful data from hardware. By keeping these issues apart, each class group may develop on its own. For instance, a recognition model may eventually be swapped out. For instance, without altering the camera or microphone control logic, a recognition model may be upgraded or changed in the future.

This package enhances flexibility and facilitates future expansion in terms of design. Future enhancements to ARIS, such environmental trigger identification or head movement detection, might be included into this package without requiring a significant reorganization of the input or command packages.

This package does not directly carry out secure transmission, dashboard-level reporting, or command validation; instead, it gives the Command Processing Package recognized outputs. This division guarantees that recognition stays more concerned with interpretation than with execution.

2.3 Command Processing and Communication Package

This package is in charge of converting identified user intent into dependable, organized, and transferable system instructions. It integrates the processes needed to provide verified outputs to the backend system with the internal command logic of ARIS.

Because it links network-based communication with local recognition, this package occupies the low-level design's core operational position. After the recognition package has determined the user

intent, the system still needs to decide whether the result is valid, whether it satisfies consistency and confidence requirements, how it maps to a predefined command entry, and how it should be sent to the backend given the current connectivity conditions.

Because of this, the package has two closely related areas of responsibility:

Command Processing Duties

- validating recognized results
- rejecting incomplete, ambiguous, or low-confidence input
- mapping valid input into a predefined structured command
- dispatching commands to the communication layer

Communication Responsibilities

- managing helmet-to-server message exchange
- maintaining active communication sessions
- detecting connection failures
- buffering command and telemetry data during interruptions
- synchronizing stored messages after reconnection

The package contains the following classes:

- **CommandValidator**
- **CommandMapper**
- **CommandDispatcher**
- **CommunicationManager**
- **ConnectionHandler**
- **MessageBuffer**

In accordance with validation standards like confidence thresholds, completeness requirements, and known-command limits, the **CommandValidator** determines if a recognized input is acceptable. Using command mappings or a command dictionary, the **CommandMapper** transforms verified

inputs into system-defined command representations. The last structured command is then sent to transmission-related components by the `CommandDispatcher`.

The package's communication component starts with `CommunicationManager`, which serves as the primary hub for coordinating incoming and outgoing communications. It collaborates with `MessageBuffer` to temporarily store messages in the event of a network outage and with `ConnectionHandler` to establish and monitor network connectivity. The communication resilience and offline support principles that were previously discussed in the earlier reports are precisely aligned with this buffering method.

Because command transformation and communication are operationally interdependent, this package is packaged together. A validated command is only helpful if it can be sent to the backend or safely stored for subsequent delivery, and a recognized result is only valuable once it has been validated and mapped. Keeping these linked classes in the same higher-level package enhances implementation clarity and traceability in a prototype like ARIS.

The internal classes continue to maintain distinct bounds at the same time. Mapping does not create network sessions, buffering does not specify the meaning of commands, and validation does not carry out encryption. The modular design objectives stated in the High-Level Design and Low-Level Design trade-off discussions are in line with this internal division.

2.4 Security, Monitoring, and Administration Package

This package is in charge of securing communication-related interactions, safeguarding system access, and facilitating operational monitoring via the web-based administrative interface. This package is mostly focused on system-wide control and backend-side visibility, in contrast to the other packages, which typically work around the helmet unit's command flow.

Its primary goal is to guarantee that ARIS operates as a safe and controllable system in addition to a recognition-and-transmission prototype. The low-level architecture must have specialized

support for authentication, authorization, encryption coordination, event recording, and dashboard-level management since ARIS handles sensitive operational and user-related data. Through RBAC, secure communication, monitoring, and privacy-oriented data processing, these issues were frequently highlighted in the previous reports.

The main responsibilities of this package are:

- authenticating users and devices before allowing communication or dashboard access
- generating and validating session-related security artifacts
- enforcing authorization policies based on user roles
- supporting encryption-related communication protection
- presenting monitoring information to operators and administrators
- managing user roles and administrative permissions
- recording system events, alerts, authentication attempts, and errors
- supporting auditability and maintenance through structured logs

The package contains the following classes:

- **AuthenticationManager**
- **EncryptionManager**
- **AccessControlManager**
- **WebInterfaceController**
- **RoleManager**
- **LogManager**

The **AuthenticationManager** ensures that users and helmet devices are verified before accessing system resources or opening communication sessions. The **EncryptionManager** supports secure data handling by coordinating encryption and decryption processes for transmitted information. The **AccessControlManager** applies authorization logic, enforcing role-based access control so that only authorized users can access protected functions.

The **WebInterfaceController** controls how system data is displayed on the dashboard from an administrative perspective. While the **LogManager** logs operational and security-related events for

monitoring and auditing reasons, the RoleManager facilitates the regulated assignment and management of system roles. The web-based interface and backend observability requirements outlined in the Analysis and High-Level Design papers are supported by these classes together.

Because security and administrative issues span several operational modules but shouldn't be directly included into them, this package is required as a stand-alone package. For instance, user-role logic should not be included in command processing, and dashboard rendering should not be handled by recognition classes. The design would become more closely connected, more difficult to test, and more difficult to maintain if all of these duties were combined into one package.

The low-level architecture maintains the distinction between control logic and operational logic by putting these classes into a separate package. As a result, the design is cleaner, allowing the security and monitoring classes to take care of system governance and observability while the core command pipeline may continue to concentrate on input interpretation and transmission.

The Recognition and Interpretation Package receives synchronized multimodal data from the Input Acquisition and Buffering Package. The Command Processing and Communication Package uses the recognized outputs generated by the Recognition and Interpretation Package. When authentication, access control, encryption coordination, or logging are needed, the Command Processing and Communication Package works with the Security, Monitoring, and Administration Package to validate, map, dispatch, buffer, and send orders. Additionally, the backend dashboard and operational visibility for administrators and command center users are supported by the Security, Monitoring, and Administration Package.

This dependence direction follows ARIS's staged execution approach and aids in maintaining low coupling. The later package is mostly in charge of safeguarding, displaying, and managing the operational data, whereas the previous packages are mainly in charge of creating it.

3. Class Interfaces

3.1 Input Processing Component

The Input Processing Component is responsible for collecting and preparing multimodal input data from the user in real time. It captures visual data through the camera and audio data through the microphone on the helmet. As the first stage of processing pipeline this component ensures that incoming data is continuously collected and temporarily stored.

3.1.1 CameraManager

Purpose:

Manages the helmet camera by initializing the device, capturing visual frames continuously, ensuring camera availability and handling camera-related failures.

Attributes:

- **cameraStatus:** Stores the current state of the camera (active, inactive or failed).
- **currentFrame:** Stores the most recently captured visual frame.

Methods:

- **initializeCamera():** Initializes the camera device.
- **startCapture():** Starts continuous frame capture.
- **stopCapture():** Stops the camera.
- **getFrame(): Frame:** Returns the latest captured frame.
- **checkCameraStatus(): Status:** Returns the current status of the camera.

Relationships:

Sends captured visual frames to InputBuffer for temporary storage and synchronization.

3.1.2 MicrophoneManager

Purpose:

Manages the microphone device by initializing audio capture, recording real-time sound input, monitoring microphone status and handling audio-related interruptions.

Attributes:

- **microphoneStatus:** Stores the current state of the microphone.
- **currentAudioChunk:** Stores the most recent audio chunk captured from the microphone.

Methods:

- **initializeMicrophone():** Initializes the microphone device.
- **startRecording():** Starts audio recording.
- **stopRecording():** Stops audio capture safely.
- **checkMicrophoneStatus(): Status:** Returns the current microphone status.
- **getAudioInput(): AudioChunk:** Returns the latest audio input.

Relationships:

Sends captured audio data to InputBuffer for temporary storage and synchronization.

3.1.3 InputBuffer

Purpose:

Temporarily stores, organizes and synchronizes visual and audio input data before forwarding structured multimodal data to the Recognition Component.

Attributes:

- **visualBuffer:** Stores recently captured visual frames.
- **audioBuffer:** Stores recently captured audio chunks.
- **latestInputPackage:** Holds the most recent synchronized multimodal input package.

Methods:

- **addVisualData(frame: Frame):** Adds a captured visual frame into the visual buffer.
- **addAudioData(audio: AudioChunk):** Adds a captured audio chunk into the audio buffer.
- **getLatestInput(): InputPackage:** Returns prepared input for processing.

Relationships:

Receives visual data from CameraManager and audio data from MicrophoneManager and then forwards synchronized input packages to the Recognition Component.

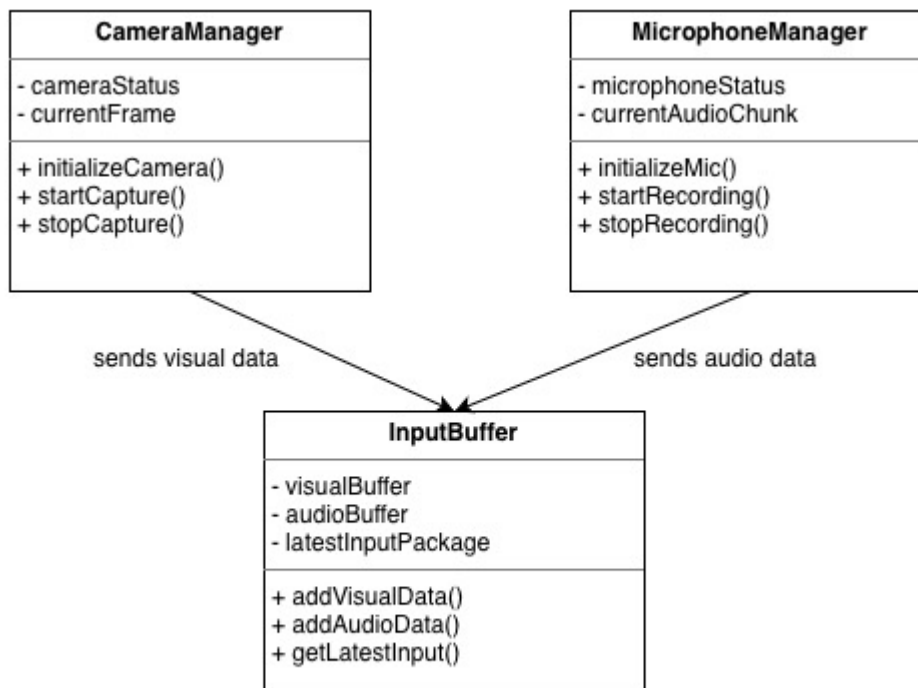


Figure : Class Diagram of Input Processing Component

3.2 Recognition Component

The Recognition Component is responsible for analyzing the processed input data and identifying user intentions using artificial intelligence techniques. It takes multimodal input from the Input Processing Component, including visual frames and audio signals. Applies recognition algorithms to detect gestures, facial expressions and voice commands.

3.2.1 GestureRecognizer

Purpose:

Detects and interprets user hand or body gestures from visual input data.

Attributes:

- **gestureModel**: Stores the trained model used for gesture recognition.
- **detectedGesture**: Holds the most recently recognized gesture.

Methods:

- **loadModel()**: Loads the pre-trained gesture recognition model.
- **recognizeGesture(frame: Frame): Gesture**: Analyzes a visual frame and returns the detected gesture.
- **updateModel()**: Updates or retrains the gesture recognition model if needed.

Relationships:

Receives visual data from InputBuffer and sends detected gestures to the Command Processing Component.

3.2.2 FaceExpressionRecognizer

Purpose:

Analyzes facial expressions from visual input to detect emotional states of the user.

Attributes:

- **faceModel**: Stores the trained model for facial expression recognition.
- **detectedExpression**: Holds the most recently recognized facial expression

Methods:

- **loadModel()**: Loads the facial recognition model.
- **recognizeExpression(frame: Frame): Expression**: Processes a visual frame and returns the detected facial expression.

- **updateModel():** Updates the model when necessary.

Relationships:

Receives visual data from InputBuffer and forwards detected expressions to the Command Processing Component.

3.2.3 VoiceCommandRecognizer

Purpose:

Processes audio input to recognize spoken commands using speech recognition techniques.

Attributes:

- **voiceModel:** Stores the trained speech recognition model.
- **recognizedCommand:** Holds the most recently recognized voice command.

Methods:

- **loadModel():** Loads the speech recognition model.
- **recognizeCommand(audio: AudioChunk): Command:** Processes audio input and returns the recognized command.
- **updateModel():** Updates the model if required.

Relationships:

Receives audio data from InputBuffer and sends recognized commands to the Command Processing Component.

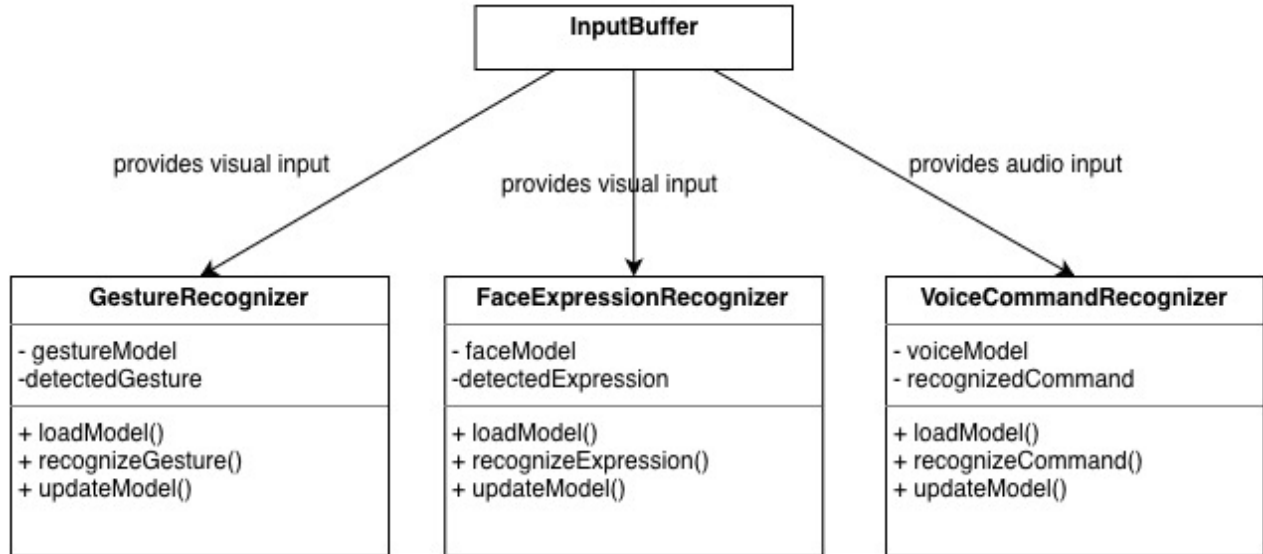


Figure : Class Diagram of Recognition Component

3.3 Command Processing Component

The Command Processing Component is responsible for converting recognized user inputs into valid system commands. It receives interpreted data such as gestures, facial expressions and voice commands from the Recognition Component and processes them through validation and mapping steps. This component ensures that only meaningful and authorized commands are executed.

3.3.1 CommandValidator

Purpose:

Ensures that recognized inputs are valid, meaningful and suitable for further processing.

Attributes:

- **validationRules:** Stores the set of rules used to validate incoming commands.
- **validationStatus:** Represents whether the current input is valid or invalid.

Methods:

- **validateCommand(input: RecognizedInput): Boolean:** Evaluates the input according to validation rules and determines its validity.

- **updateRules():** Updates validation rules if needed.

Relationships:

Receives recognized inputs from the Recognition Component and forwards valid inputs to the CommandMapper.

3.3.2 CommandMapper

Purpose:

Converts validated inputs into corresponding system commands based on predefined mappings.

Attributes:

- **commandMap:** Stores mappings between recognized inputs and system commands.
- **mappedCommand:** Holds the command generated from the mapping process.

Methods:

- **mapCommand(input: RecognizedInput): Command:** Converts validated input into a corresponding system command.
- **updateMapping():** Updates command mappings when necessary.

Relationships:

Receives validated inputs from CommandValidator and forwards mapped commands to the CommandDispatcher.

3.3.3 CommandDispatcher

Purpose:

Handles the execution and forwarding of commands to the appropriate system components.

Attributes:

- **currentCommand:** Stores the command currently being processed.

- **dispatchStatus:** Indicates whether the command has been successfully dispatched.

Methods:

- **dispatchCommand(command: Command):** Sends the command to the Communication Component.
- **trackStatus(): Status:** Monitors the execution or transmission status of the command.

Relationships:

Receives mapped commands from CommandMapper and forwards them to the Communication Component.

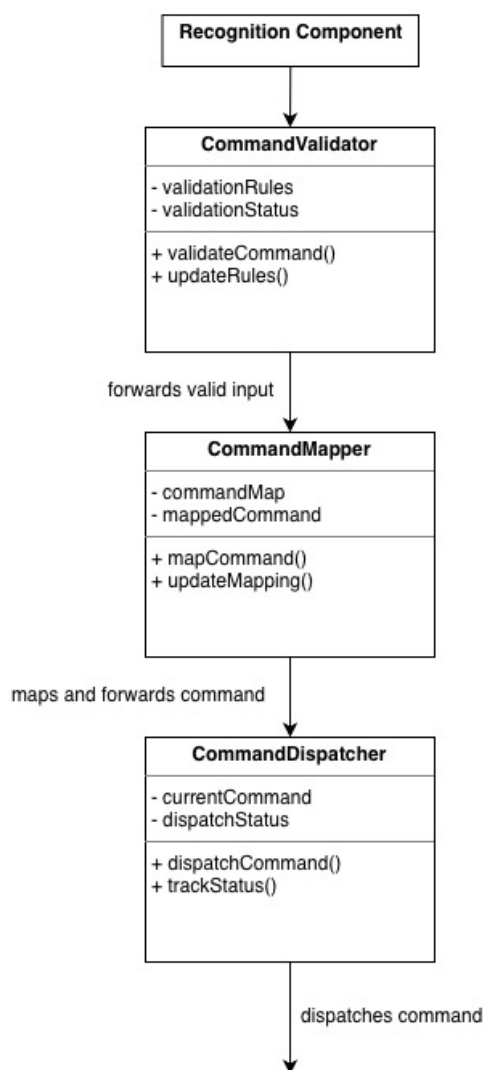


Figure : Class Diagram of Command Processing Component

3.4 Communication Component

The Communication Component is responsible for managing reliable and continuous data exchange between the ARIS helmet unit and the central server. This component handle the transmission of validated commands, telemetry data and also feedback messages through a secure wireless channel. It also supports two way communication and temporary offline operation by buffering messages when connectivity is lost and synchronizing them after reconnection.

3.4.1 CommunicationManager

Purpose:

The CommunicationManager functions as the central system that manage all passing information between the helmet unit and the central server. This system operates to send verified commands together with telemetry data and feedback messages to their correct destinations using an orderly transmission system.

Attributes:

- **connectionStatus:** Current state of connection (connected/disconnected)
- **activeSessionID:** Identifier of current communication session
- **outgoingQueue:** Queue for messages to be sent
- **incomingQueue:** Queue for received messages
- **transmissionProtocol:** Communication protocol used (TCP/WebSocket)

Methods:

- **sendCommand(commandData):** Send validated command to server
- **receiveMessage():** Receive incoming data from server
- **sendTelemetry(data):** Transmits system status information
- **forwardFeedback(data):** Forward feedback to UI or system
- **synchronizeBufferedMessages():** Send stored messages after reconnection

Relationships:

- The system establishes and maintains its communication link through `ConnectionHandler`, which enables the link to be restored after a connection interruption.
- The system uses `MessageBuffer` to keep messages in storage until network access is restored.
- The system transmits sensitive data after establishing a connection with the Security Component, which includes `EncryptionManager`

3.4.2 `ConnectionHandler`

Purpose:

The `ConnectionHandler` manages the communication session between the helmet unit and the central server. The interface system needs to establish network connections while checking their status and identifying any connection problems which require automatic reconnection.

Attributes:

- **serverAddress:** Address of the communication server
- **portNumber:** Port used for communication
- **protocolType:** Type of protocol (e.g., TCP/UDP)
- **retryCount:** Number of reconnection attempts
- **timeoutDuration:** Allowed time before connection timeout

Methods:

- **establishConnection():** Initiates connection with the server
- **verifyConnection():** Checks if the connection is active
- **reconnect():** Attempts to restore lost connection
- **terminateConnection():** Closes the connection safely
- **detectFailure():** Detects connection or transmission failure

Relationships:

- The system functions under the management of CommunicationManager.
- The system send alerts to CommunicationManager whenever there is a change in the connection status.
- The system indirectly support MessageBuffer providing synchronization capabilities which become active after a user disconnects and then reconnects.
- The system works together with AuthenticationManager to establish secure session initiation for their authentication process.

3.4.3 MessageBuffer

Purpose:

The MessageBuffer temporarily stores validated commands and telemetry data when the communication channel is interrupted. This class supports fault tolerance by preventing immediate data loss and allowing delayed synchronization once the connection is restored.

Attributes:

- **bufferedMessages:** List of temporarily stored messages
- **bufferCapacity:** Maximum size of the buffer
- **timestampList:** Timestamps of stored messages
- **syncStatus:** Indicates synchronization state

Methods:

- **storeMessage(messageData):** Adds a message to the buffer
- **retrieveMessages():** Retrieves stored messages
- **clearMessages():** Removes all messages from buffer
- **checkCapacity():** Checks if buffer limit is reached
- **markAsSynced():** Marks messages as successfully delivered

Relationships:

- The CommunicationManager use this system when its operators need to send information but cannot achieve immediate delivery.

- The system maintains data storage operations until the ConnectionHandler confirm that network connectivity has been restored.
- The system transmits all stored messages to CommunicationManager which will use them to execute retransmission and synchronization tasks.

Within this component CommunicationManager acts as the central coordinator, ConnectionHandler manages network session continuity, and MessageBuffer supports temporary offline storage. Together these classes ensure that ARIS can maintain reliable communication even under unstable network conditions.

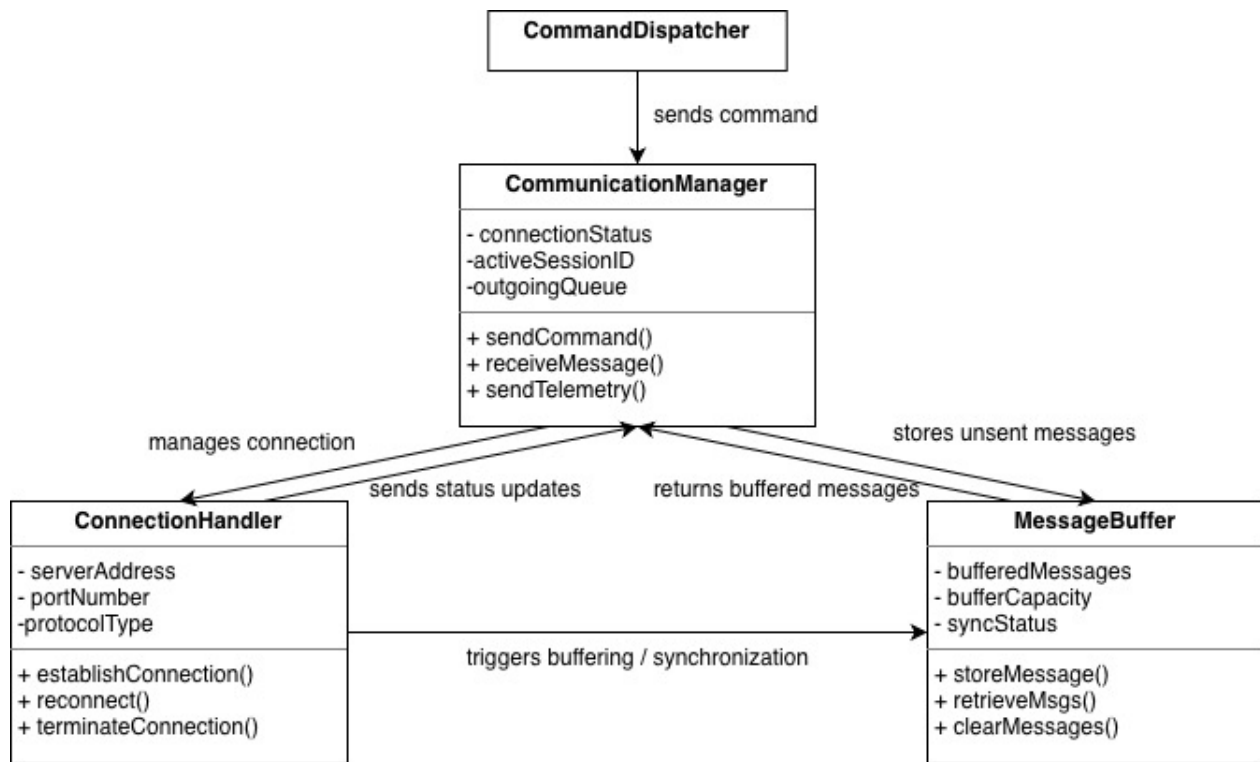


Figure : Class Diagram of Communication Component

3.5 Security Component

The Web Interface and Administration Component provides the web based dashboard and administrative functions required for monitoring, access management and also system visibility in ARIS. The system present its current operational state and connection details and active alerts and past data records through an interface that enables users to access information. The system enables administrative control through two functions which include role management and log tracking.

3.5.1 AuthenticationManager

Purpose:

The AuthenticationManager uses user identity verification together with helmet device authentication to control access to system resources and communication services. The system permit session initiation and continuation to trusted user who have successfully authenticated themselves.

Attributes:

- **registeredDeviceList:** List of authorized helmet devices
- **userCredentialStore:** Stored login credentials for web users
- **authenticationStatus:** Current result of authentication process
- **sessionToken:** Token assigned to authenticated session
- **failedAttemptCount:** Number of unsuccessful login attempts

Methods:

- **authenticateDevice(deviceID, credentialData):** Verifies helmet identity before communication
- **authenticateUser(username, password):** Validates web user login information
- **generateSessionToken():** Creates a token for a valid session
- **validateSessionToken(token):** Checks whether a session token is still valid
- **rejectUnauthorizedAccess():** Blocks access when authentication fails

Relationships:

- The application establish partnership with ConnectionHandler to initiate secure session connections.
- The system transfers confirmed identity details to AccessControlManager which uses them to determine user access rights.
- The system provides user access validation to WebInterfaceController to control dashboard access.

3.5.2 EncryptionManager

Purpose:

The EncryptionManager protects the confidentiality and integrity of transmitted data by applying encryption and decryption operations during communication between the helmet unit and the central server.

Attributes:

- **encryptionAlgorithm:** Algorithm used for data protection
- **encryptionKey:** Current key used for encryption and decryption
- **keyVersion:** Version identifier of the active key
- **encryptionStatus:** Status of encryption process

Methods:

- **encryptData(plainData):** Converts readable data into encrypted form
- **decryptData(cipherData):** Restores encrypted data into readable form
- **generateKey():** Creates a new encryption key
- **updateKey():** Renews the current key when needed
- **verifyDataIntegrity(data):** Checks whether transmitted data was altered

Relationships:

- The system operate under its initial state protection until CommunicationManager transmits sensitive information.
- The system establish secure authenticated sessions through its collaboration between AuthenticationManager and another security mechanism.
- The system safeguards command signal together with telemetry information and feedback data which the system transmits.

3.5.3 AccessControlManager

Purpose:

The AccessControlManager enforces authorization rules by checking which system resources and operations are allowed for each user role. The system enables secure administrative functions through its design which restricts user access to unauthorized system capabilities.

Attributes:

- **roleList:** List of defined system roles
- **permissionMatrix:** Mapping between roles and allowed actions
- **currentUserRole:** Active role of authenticated user
- **accessRules:** Authorization rules applied in the system

Methods:

- **assignRole(userID, roleType):** Assigns a role to a user
- **checkPermission(userID, action):** Verifies whether a user can perform an action
- **grantAccess(userID, resource):** Allows access to an authorized resource
- **denyAccess(userID, resource):** Blocks access to a restricted resource
- **updatePermissions(roleType, permissionSet):** Changes allowed permissions of a role

Relationships:

- The system initiate its operations after AuthenticationManager completes the process of verifying user identities.
- The system delivers the authorization results to WebInterfaceController which requires them.
- The system enables users with administrative rights to manage their access permissions through integration with RoleManager.

The component shows three different elements which include AuthenticationManager for identity verification, EncryptionManager for data protection during transmission and also AccessControlManager for implementing role-based access control regulations. The classes work

together to protect ARIS communication and administration functions from unauthorized access and insecure data exchange.

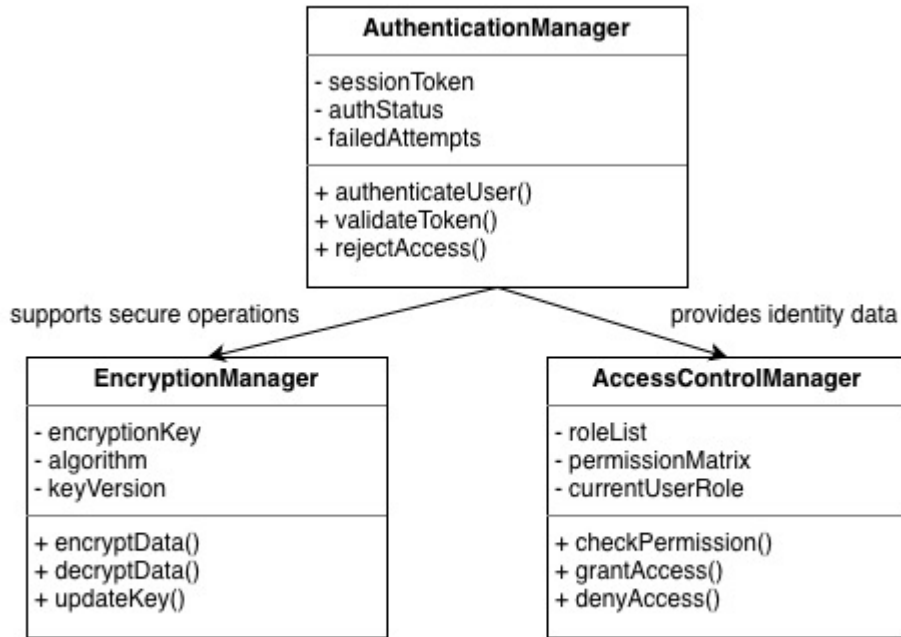


Figure : Class Diagram of Security Component

3.6 Web Interface and Administration Component

The Web Interface and Administration Component provides the web based dashboard and administrative functions required for monitoring, access management and also system visibility in ARIS. The system presents its current operational state and active connections and generated alerts and past data to users who have permission to access the system through a user friendly display. The system enables administrators to control operations by managing user roles and monitoring log activities.

3.6.1 WebInterfaceController

Purpose:

The WebInterfaceController oversees the operation of the web dashboard used by command center

operators and administrators. The system show helmet connection status together with command flow information and alert and system status through an understandable display system.

Attributes:

- **activeDashboardSessions:** Currently active dashboard users
- **displayedHelmetList:** List of connected helmets shown on interface
- **currentSystemStatus:** Latest status information displayed on dashboard
- **alertPanelData:** Active alerts shown to authorized users
- **connectionTimeInfo:** Connection duration and timing information

Methods:

- **renderDashboard():** Loads the main dashboard view
- **updateSystemStatus(statusData):** Updates displayed system information
- **displayAlert(alertData):** Shows alert messages on the interface
- **showConnectionInfo():** Presents connection and session details
- **refreshHelmetView():** Refreshes connected helmet data on screen

Relationships:

- The system employ RoleManager to control which interface components users with different roles are allowed to see.
- The system uses LogManager to fetch historical and event records when it needs to show log information.
- The system operates with the Security Component to provide users who have been authenticated and authorized access to the dashboard.

3.6.2 RoleManager

Purpose:

The RoleManager controls user role functions within the ARIS system by managing three different

user types which include root users, command center operator and also standard users. The system enable administrators to create user permissions while they control access rights for users.

Attributes:

- **roleDefinitions:** List of available system roles
- **assignedUsers:** Mapping of users to roles
- **roleHierarchy:** Priority structure among roles
- **editablePermissionSet:** Permissions that can be configured by administrators

Methods:

- **createRole(roleName):** Creates a new system role
- **assignUserRole(userID, roleName):** Assigns a role to a user
- **removeUserRole(userID):** Removes a role assignment
- **updateRoleDefinition(roleName, permissionSet):** Modifies permissions of a role
- **getRoleInformation(roleName):** Returns details of a selected role

Relationships:

- WebInterfaceController need role and permission details which this system provides.
- AccessControlManager works together with this system to ensure that authorization rules remain uniform.
- The system enables root level users to make changes for user access rights and permission management.

3.6.3 LogManager

Purpose:

The LogManager records system events which need to be preserved for monitoring purposes and diagnostic work and maintenance activities. The system maintains logs of command transmissions and alerts and authentication events and system errors.

Attributes:

- **logEntries:** Collection of recorded system events
- **logStorageLocation:** Storage path or database location of logs
- **eventType:** Category of logged event
- **timestamp:** Time of recorded event
- **retentionPolicy:** Rule for log retention and deletion

Methods:

- **recordEvent(eventData):** Stores a new event in the log system
- **storeSystemLog(logData):** Saves detailed system log information
- **retrieveLogs(filterCriteria):** Returns logs based on filters
- **archiveLogs():** Archives older log records
- **deleteExpiredLogs():** Removes outdated log data according to policy

Relationships:

- The system sends log data and monitoring information to WebInterfaceController.
- The system receives event information through its communication and authentication and system monitoring processes.
- The system assists administrators and operators with their maintenance tasks and audit processes.

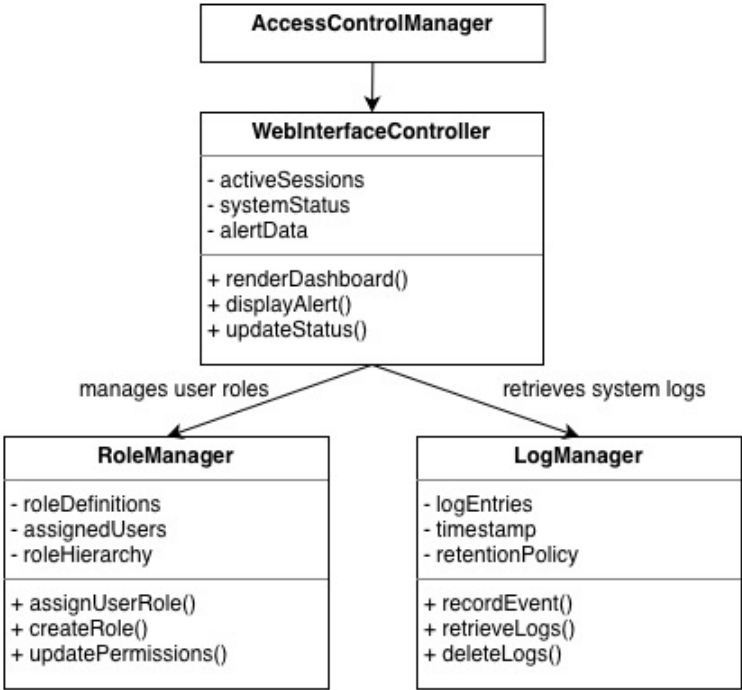


Figure : Class Diagram of Web Interface Component

The **WebInterfaceController** functions as the presentation layer of this component while **RoleManager** manages administrative role distribution and **LogManager** tracks operational activities for monitoring purposes and analysis. The three classes work together to create the dashboard which provides necessary administrative functions needed to maintain secure and functional ARIS operations.

4. Glossary

ARIS (AI-Assisted Smart Helmet Communication System): ARIS is a smart helmet system that enables hands-free communication by interpreting user inputs like gestures, facial expressions, and voice commands.

Multimodal Input: Using multiple input types (such as voice, gestures, and facial expressions) together to improve accuracy and reliability.

Helmet Unit: The physical device that collects user inputs and interacts with the system.

Command: A predefined action generated after processing user input.

Recognition Component: Analyzes incoming data and identifies what the user wants to do.

Command Processing Component: Checks recognized inputs and converts them into executable system commands.

CommandValidator: Ensures that the input is valid, complete, and meaningful before processing.

CommandMapper: Converts validated inputs into predefined system commands.

Communication Component: Handles the communication between the helmet and the backend system.

CommunicationManager: Manages data exchange between different system components.

Session Management: Maintains active connections and manages communication during system operation.

Telemetry Data: Includes system information like status, connection details, and performance data for monitoring.

Backend Server: The central system that processes commands, manages sessions, and stores system data.

Connection Handler: Manages establishing and maintaining the connection between system components.

Input Synchronization: The process of aligning multimodal inputs (audio and video) in time to ensure consistent and accurate processing by recognition components.

Input Buffering: Temporary storage of captured input data before it is processed, enabling smooth data flow and handling of asynchronous inputs.

Edge Device: A computing unit located close to the data source (helmet unit) that performs local processing to reduce latency and dependency on network connectivity.

Recognition Output: The interpreted result produced by recognition components representing a possible user command along with its associated confidence level.

Confidence Threshold: A predefined minimum confidence score required for a recognized input to be considered valid and processed further.

Command Validation: The process of verifying whether a recognized input is complete, reliable, and meets predefined system constraints before execution.

Command Mapping: The process of translating validated user inputs into predefined system commands using a command dictionary.

Command Dispatching: The process of forwarding validated and mapped commands to the communication subsystem for transmission.

Command Transmission: The process of sending validated commands from the helmet unit to the backend system through secure communication channels.

Multimodal Synchronization: Combining and aligning different input modalities (gesture, voice, facial expression) to improve recognition accuracy and system robustness.

Connection Recovery: The process of restoring communication between system components after a connection failure and resuming data transmission.

Offline Mode: A temporary operational state where the system continues functioning locally and stores data when network connectivity is unavailable.

Data Buffering (Communication): Temporary storage of commands and telemetry data during network interruptions to ensure no data loss.

Session State: The current status of an active communication session, including connection validity, authentication state, and activity tracking.

Secure Session: A communication session established using authentication and encryption to ensure confidentiality and integrity of transmitted data.

System Monitoring: The process of observing system performance, status, and operational metrics through logs and dashboard interfaces.

Operational Visibility: The ability of administrators and operators to observe real-time system status, commands, and alerts.

Administrative Control: The ability to manage system configurations, user roles, and operational settings through authorized interfaces.

Auditability: The capability of the system to track and review historical events, logs, and actions for analysis and security purposes.

Modularity: A design principle where the system is divided into independent components that can be developed and maintained separately.

Low Coupling: A system design property where components have minimal dependencies on each other, improving flexibility and maintainability.

5. References

1. ACM, *ACM Code of Ethics and Professional Conduct*, 2018.
Available: <https://www.acm.org/code-of-ethics>
2. B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns, and Java*, 2nd ed., Prentice Hall, 2004.
3. IEEE, *IEEE Standard for Software Design Descriptions (IEEE 1016-2009)*, IEEE Computer Society, 2009.
4. I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.
5. ARIS Project Team, *Analysis Report*, TED University, 2025.
6. ARIS Project Team, *High-Level Design Report*, TED University, 2025.
7. SENG 214 Software Engineering, Lecture Notes, TED University, 2025.